

云计算技术课程设计报告

Cloud Computing Technologies Course Project

题目：基于华为云 CCE 的容器化应用部署与 Spark 大数据分析

课程名称	云计算技术
课程代码	SCAI004712
学生姓名	李浩洋、黄腾洋
学号	2023112430、2023112431
班级	软件 2023-3 班、计算机 2023-1 班
实验平台	华为云 CCE / SWR / ELB / EVS
提交日期	2026 年 6 月

代码仓库链接：<https://github.com/ayukas16/cloud-course-design.git>

小组分工：

小组成员	工作内容	工作量占比
李浩洋 (2023112430)	完成前后端容器化改造、K8s 全套资源编排；完成 Spark Operator 部署、PySpark 数据清洗策略制定、4 项统计分析代码编写及 Pandas/PySpark 性能对比实验。	50%
黄腾洋 (2023112431)	协助完成华为云资源申请与释放；负责执行 kubectl 验证命令、运行压测脚本并截取关键状态图；负责 Spark 数据集的上传与路径配置；负责报告最终排版、图表编号。	50%
公共协作	阶段性进度对齐、实验结果交叉验证、答辩材料准备。	共同

目录

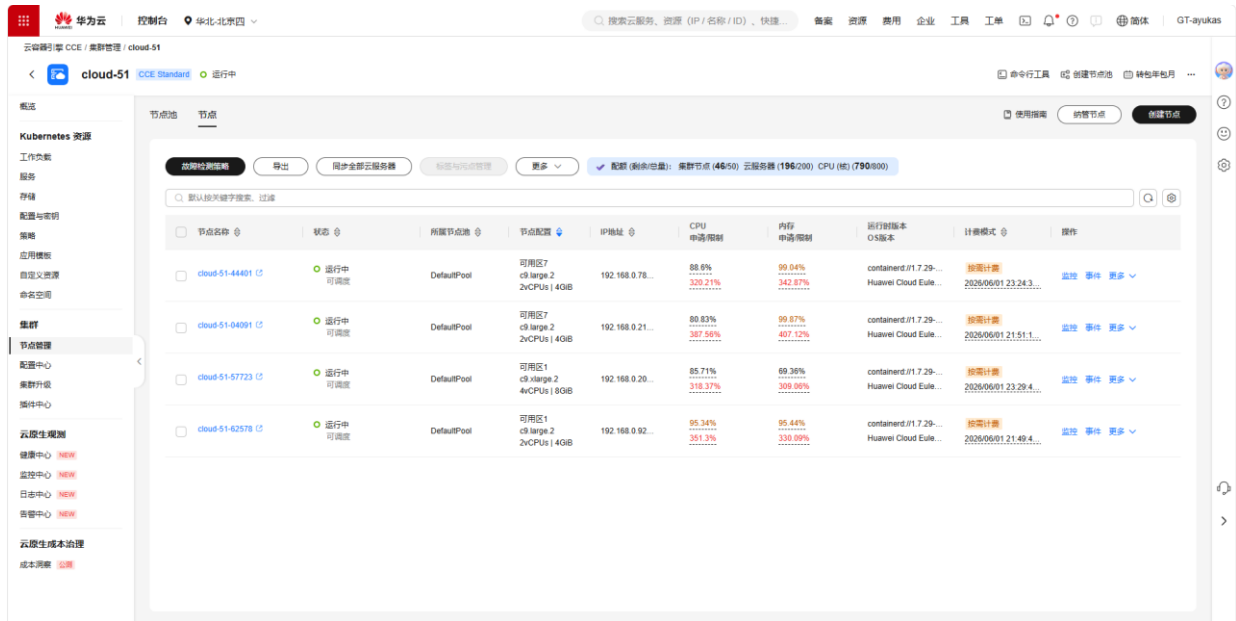
1 华为云环境信息.....	2
2 第一部分：云计算平台搭建.....	4
2.1 任务 1：应用容器化.....	4
2.2 任务 2：CCE 集群搭建.....	5
2.3 任务 3：应用部署.....	6
2.4 任务 4：持久化存储.....	8
2.5 任务 5：ConfigMap Volume 挂载.....	9
2.6 任务 6：HPA 弹性伸缩.....	11
3 第二部分：方向 A Spark 大数据分析.....	13
3.1A-0：Spark Operator 环境部署.....	13
3.2A-1：数据清洗.....	15
3.3A-2：SparkSQL 统计分析.....	19
3.3.1 查询 1：按类型统计电影数量和平均评分.....	19
3.3.2 查询 2：评分最高 Top 10 电影.....	20
3.3.3 查询 3：按年份统计电影数量和平均评分.....	21
3.3.4 查询 4：窗口函数选取每年评分最高电影.....	22
3.4A-3：性能对比与 Amdahl 分析.....	23
4 总结与收获.....	25
5 附录：核心配置与代码片段.....	25
5.1 后端 Deployment 关键配置.....	25
5.2 Redis PVC 关键配置.....	26
5.3SparkApplication 分析作业关键配置.....	26
5.4analysis.py 核心清洗逻辑.....	26

1 华为云环境信息

本课程设计在华为云 cn-north-4 区域完成。实验使用 SWR 存放镜像，使用 CCE 承载 Kubernetes 工作负载，使用 ELB 暴露后端服务，使用 EVS/csi-disk 为 Redis 提供持久化存储。第二部分选择方向 A，即在同一 CCE 集群上通过 Spark Operator 运行 PySpark 作业。

项目	最终配置
Region	cn-north-4

SWR 组织	cloud-51
SWR 地址	swr.cn-north-4.myhuaweicloud.com/cloud-51
CCE 集群	cloud-51
Kubernetes 版本	不低于 1.27，实际以 kubectl get nodes -o wide 截图为准
网络插件	YangtseCNI
存储类	csi-disk
第二部分方向	方向 A：Spark 大数据分析



节点名称	状态	所属节点池	节点配置	IP地址	CPU 申请/限制	内存 申请/限制	运行时版本 OS版本	计费模式	操作
cloud-51-44401	运行中 可调度	DefaultPool	可用区7 c6.large.2 2vCPUs 4GiB	192.168.0.78...	86.6% 320.21%	99.04% 342.87%	containerd/1.7.29-... Huawei Cloud Eule...	按量计费 2025/06/01 23:24.3...	监控 事件 更多
cloud-51-04091	运行中 可调度	DefaultPool	可用区7 c6.large.2 2vCPUs 4GiB	192.168.0.21...	80.83% 367.56%	99.87% 407.12%	containerd/1.7.29-... Huawei Cloud Eule...	按量计费 2025/06/01 21:51.1...	监控 事件 更多
cloud-51-57723	运行中 可调度	DefaultPool	可用区1 c6.large.2 4vCPUs 8GiB	192.168.0.20...	85.71% 318.37%	69.36% 309.96%	containerd/1.7.29-... Huawei Cloud Eule...	按量计费 2025/06/01 23:29.4...	监控 事件 更多
cloud-51-62578	运行中 可调度	DefaultPool	可用区1 c6.large.2 2vCPUs 4GiB	192.168.0.92...	95.34% 351.3%	95.44% 330.09%	containerd/1.7.29-... Huawei Cloud Eule...	按量计费 2025/06/01 21:49.4...	监控 事件 更多

图 1-1CCE 集群节点状态（kubectl get nodes -o wide）

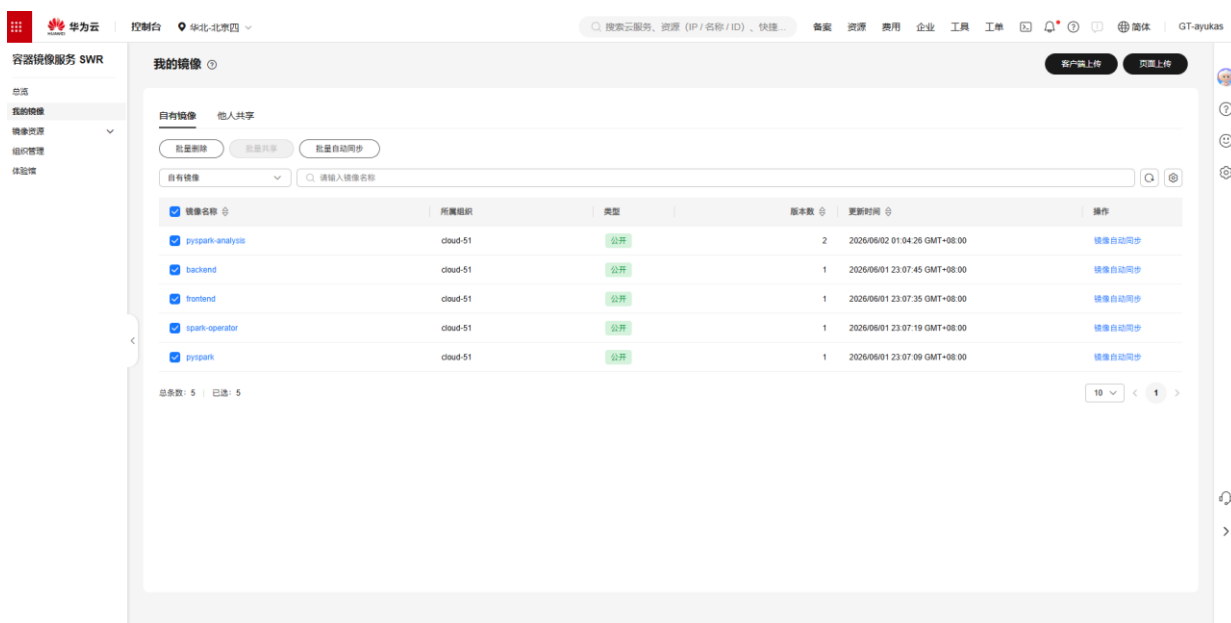


图 1-2 华为云 SWR 镜像仓库列表

2 第一部分：云计算平台搭建

第一部分完成 Flask 后端 API、Nginx 前端和 Redis 的容器化部署。后端通过 Redis 记录/api/ping 访问次数，前端通过 Nginx 代理访问后端接口。Kubernetes 侧将后端、Redis、Service、ConfigMap、Secret、PVC、HPA 等资源分离配置。

2.1 任务 1：应用容器化

后端使用 Python Flask 实现 API 服务，Redis 用于保存访问计数。Dockerfile.backend 保留多阶段构建结构，requirements.txt 中包含 flask、redis 和 requests，其中 requests 作为自选 Python 包加入依赖。前端使用 Nginx 镜像托管静态页面，index.html 中包含学号、姓名和接口测试按钮。

镜像	Tag	说明
backend	swr.cn-north-4.myhuaweicloud.com/cloud-51/backend:v1	Flask 后端 API 镜像
frontend	swr.cn-north-4.myhuaweicloud.com/cloud-51/frontend:v1	Nginx 前端静态页镜像
redis	redis:7-alpine	Redis 官方镜像，K8s 中配置密码

		和持久化
--	--	------

本地联调使用 docker compose up --build 启动 backend、frontend 和 redis。访问前端页面后，点击请求按钮或直接访问/api/ping，后端返回 status 为 ok 的 JSON，说明前后端和 Redis 通信正常。

```
docker compose up --build
curl http://localhost:5000/api/ping

docker buildx build --provenance=false --platform linux/amd64 -t swr.cn-north-4.myhuaweicloud.com/cloud-51/backend:v1 --push .\backend
docker buildx build --provenance=false --platform linux/amd64 -t swr.cn-north-4.myhuaweicloud.com/cloud-51/frontend:v1 --push .\frontend
```



云计算课程设计

姓名：李浩洋 黄腾洋

学号：2023112430 2023112431

测试后端 API

```
{
  "count": 9991,
  "message": "backend connected redis",
  "status": "ok"
}
```

图 2-1 前端返回结果

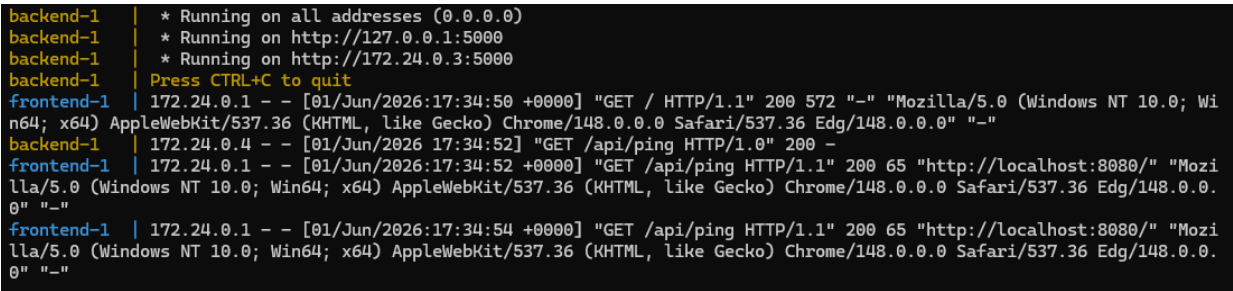


图 2-2 后端返回结果

2.2 任务 2：CCE 集群搭建

CCE 集群采用云上托管 Master 节点，学生侧创建 Worker 节点。集群创建完成后下载公网 kubeconfig，并在本地 Windows CMD 中通过 KUBECONFIG 指定配置文件。kubectl get nodes -o wide 显示所有 Worker 节点均为 Ready，且 VERSION 列满足课程要求。

```
cd /d F:\云计算技术\cloud-course-design
set KUBECONFIG=F:\云计算技术\cloud-course-design\kubeconfig-cloud-51-public.yaml
kubectl get nodes -o wide
kubectl get storageclass
```

```
F:\云计算技术\cloud-course-design>kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE     VERSION             INTERNAL-IP    EXTERNAL-IP    OS-IMAGE
192.168.0.205 Ready     <none>    119m    v1.34.3-r0-34.0.12.3 192.168.0.205 <none>         Huawei Cloud EulerOS 2.0
(x86_64) 5.10.0-182.0.0.95.r3353_273.hce2.x86_64 containerd://1.7.29-11-g0a540c3d5
192.168.0.217 Ready     <none>    3h38m   v1.34.3-r0-34.0.12.3 192.168.0.217 <none>         Huawei Cloud EulerOS 2.0
(x86_64) 5.10.0-182.0.0.95.r3090_252.hce2.x86_64 containerd://1.7.29-11-g0a540c3d5
192.168.0.78 Ready     <none>    124m    v1.34.3-r0-34.0.12.3 192.168.0.78 <none>         Huawei Cloud EulerOS 2.0
(x86_64) 5.10.0-182.0.0.95.r3353_273.hce2.x86_64 containerd://1.7.29-11-g0a540c3d5
192.168.0.92 Ready     <none>    3h39m   v1.34.3-r0-34.0.12.3 192.168.0.92 <none>         Huawei Cloud EulerOS 2.0
(x86_64) 5.10.0-182.0.0.95.r3090_252.hce2.x86_64 containerd://1.7.29-11-g0a540c3d5
```

图 2-3kubectl get nodes -o wide 验证节点 Ready

2.3 任务 3：应用部署

应用部署阶段在 default 命名空间创建后端 Deployment、Redis Deployment、后端 LoadBalancer Service、Redis ClusterIP Service、ConfigMap 和 Secret。后端副本数设置为 2，并配置 CPU/内存 requests 与 limits；Redis 副本数为 1，内存上限不超过 512Mi。ConfigMap 注入 REDIS_HOST 和 REDIS_PORT，Secret 注入 Redis 密码。

资源	名称	关键配置
Deployment	backend	replicas=2; image=cloud-51/backend:v1; resources; envFrom ConfigMap; secretKeyRef
Deployment	redis	replicas=1; image=redis:7-alpine; limits.memory=256Mi
Service	backend-svc	LoadBalancer; 端口 80 转发到 5000; ELB union 注解
Service	redis-svc	ClusterIP; 端口 6379, 仅集群内部访问
ConfigMap	backend-config	REDIS_HOST=redis-svc; REDIS_PORT=6379; APP_ENV=production
Secret	redis-secret	Redis 密码 redis123456 的 base64 编码 cmVkaXMxMjM0NTY=

```
kubectl apply -f k8s\configmap-secret.yaml
kubectl apply -f k8s\redis-deployment.yaml
kubectl apply -f k8s\backend-deployment.yaml
kubectl apply -f k8s\service.yaml

kubectl get pods -o wide
kubectl get svc backend-svc redis-svc
curl http://<backend-svc-EXTERNAL-IP>/api/ping
```

最终访问后端/api/ping 接口返回 status 为 ok 的 JSON，说明后端 Pod、Redis Service、ConfigMap 与 Secret 均生效。

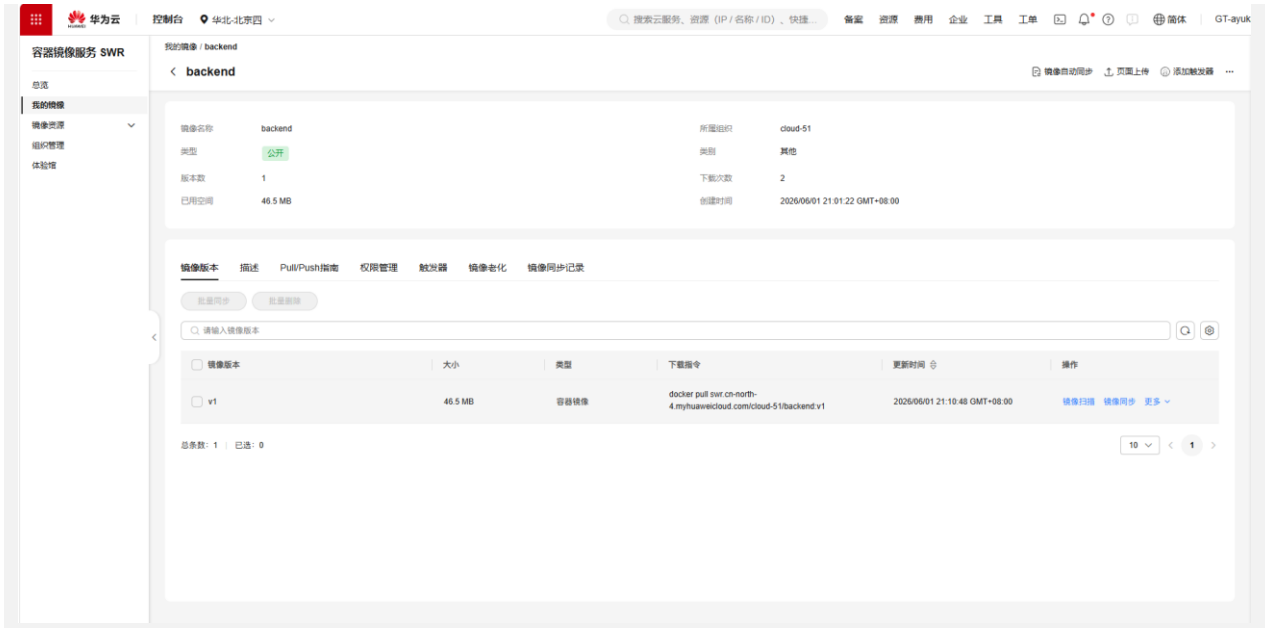


图 2-4backend 与 redis Pod 运行状态

```
F:\云计算技术\cloud-course-design>kubectl get deploy backend redis
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
backend   2/2     2             2            88m
redis     1/2     1             1            137m

F:\云计算技术\cloud-course-design>kubectl get pods -l "app in (backend,redis)" -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE           NOMINATED NODE   READINESS GATES
backend-6769558f75-csbc5            1/1     Running   0           2m23s  10.0.1.141    192.168.0.205   <none>            <none>
backend-6769558f75-nlt7x            1/1     Running   0           89m    10.0.1.137    192.168.0.205   <none>            <none>
redis-79dd54d49c-8z7dc              0/1     Pending   0           91m    <none>        <none>          <none>            <none>
redis-846b8799d9-lpcts              0/1     Pending   0           91m    <none>        <none>          <none>            <none>
redis-846b8799d9-s2qkg             1/1     Running   0           102m   10.0.0.148    192.168.0.217   <none>            <none>

F:\云计算技术\cloud-course-design>kubectl get svc backend-svc redis-svc
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
backend-svc  LoadBalancer  10.247.219.75  113.44.218.176,192.168.0.187  80:31807/TCP    3h38m
redis-svc    ClusterIP      10.247.201.163  <none>        6379/TCP         3h38m

F:\云计算技术\cloud-course-design>kubectl get configmap backend-config
NAME      DATA   AGE
backend-config  3       3h38m

F:\云计算技术\cloud-course-design>kubectl get secret redis-secret
NAME      TYPE   DATA   AGE
redis-secret  Opaque  1       3h38m

F:\云计算技术\cloud-course-design>curl http://113.44.218.176/api/ping
{"count":404,"message":"backend connected redis","status":"ok"}
```

图 2-5 通过 ELB 公网 IP 访问/api/ping

注：该图中处于 pending 的节点是进行后续实验时的实验节点，
此时环境已恢复至任务三，所以该节点状态为 pending

2.4 任务 4：持久化存储

Redis 使用 PVC 挂载/data 目录，PVC 的 storageClassName 为 csi-disk，容量为 10Gi，访问模式为 ReadWriteOnce。Redis 开启 appendonly 持久化，写入 testkey 后删除 Redis Pod，Deployment 自动重建新 Pod，新 Pod 仍能读取到 testkey，说明 Redis 数据未 Pod 删除而丢失。

```
kubectl apply -f k8s\redis-pvc.yaml
kubectl apply -f k8s\redis-deployment.yaml
kubectl get pvc
```

```
kubectl exec -it <redis-pod> -- redis-cli -a redis123456 SET testkey "hello"
kubectl exec -it <redis-pod> -- redis-cli -a redis123456 GET testkey
kubectl delete pod <redis-pod>
kubectl exec -it <new-redis-pod> -- redis-cli -a redis123456 GET testkey
```

```
F:\云计算技术\cloud-course-design>kubectl get pvc
NAME                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTR
ibutesclass         AGE
redis-data-pvc      Bound    pvc-9f380309-7106-4c5c-a0d9-483dc82d6dcd  10Gi        RWO             csi-disk       <unset>
154m

F:\云计算技术\cloud-course-design>kubectl get pods | findstr redis
redis-846b8799d9-tfpp6      1/1      Running    0          57s

F:\云计算技术\cloud-course-design>kubectl exec -it redis-846b8799d9-tfpp6 -- redis-cli -a redis123456 SET testkey "hello"
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
OK

F:\云计算技术\cloud-course-design>kubectl exec -it redis-846b8799d9-tfpp6 -- redis-cli -a redis123456 GET testkey
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"hello"
```

图 2-6 Redis 写入 testkey

```
F:\云计算技术\cloud-course-design>kubectl delete pod redis-846b8799d9-tfpp6
pod "redis-846b8799d9-tfpp6" deleted from default namespace
```

图 2-7 删除 Redis Pod 后自动重建

```
F:\云计算技术\cloud-course-design>kubectl exec -it redis-846b8799d9-mzttv -- redis-cli -a redis123456 GET testkey
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"hello"
```

图 2-8 新 Redis Pod 中 GET testkey 返回 hello

2.5 任务 5: ConfigMap Volume 挂载

任务 5 在任务 3 的前端 Deployment 基础上修改，不重新构建前端镜像。新增 nginx-config ConfigMap，将完整 default.conf 写入 data 字段；前端 Deployment 通过 volumeMounts 将 ConfigMap 挂载到/etc/nginx/conf.d/default.conf 或/etc/nginx/conf.d 目录。修改 ConfigMap 中的 proxy_pass 端口后，重新 apply 并进入前端 Pod 查看配置文件，验证容器内文件内容已更新。

```
kubectl apply -f k8s\nginx-configmap.yaml
kubectl apply -f k8s\frontend-deployment.yaml
kubectl get pods | findstr frontend
kubectl exec -it <frontend-pod> -- cat /etc/nginx/conf.d/default.conf
```

envFrom 适合注入简单键值型配置，例如 Redis 地址、端口和运行环境。ConfigMap Volume 适合注入结构化、多行配置文件，例如 Nginx 的 default.conf。相比环境变量，Volume 挂载更适合由程序按文件路径读取的配置文件，也便于统一管理和替换。

```

1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: nginx-config
5    namespace: default
6  data:
7    default.conf: |
8      server {
9        listen 80;
10
11        location / {
12          root /usr/share/nginx/html;
13          index index.html;
14        }
15
16        location /api/ {
17          proxy_pass http://backend-svc:80;
18        }
19      }

```

图 2-9nginx-config ConfigMap 内容

```

F:\云计算技术\cloud-course-design>kubectl apply -f k8s\nginx-configmap.yaml
configmap/nginx-config unchanged

F:\云计算技术\cloud-course-design>kubectl apply -f k8s\frontend-deployment.yaml
deployment.apps/frontend unchanged

F:\云计算技术\cloud-course-design>kubectl scale deployment frontend --replicas=1
deployment.apps/frontend scaled

F:\云计算技术\cloud-course-design>kubectl rollout status deployment/frontend
deployment "frontend" successfully rolled out

F:\云计算技术\cloud-course-design>kubectl get pods | findstr frontend
frontend-d6996cd5f-ngwhz          1/1      Running   0          2m48s

F:\云计算技术\cloud-course-design>kubectl exec -it frontend-d6996cd5f-ngwhz -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc:80;
    }
}

```

图 2-10 前端 Pod 内 default.conf 挂载结果

```

F:\云计算技术\cloud-course-design>kubectl rollout restart deployment/frontend
deployment.apps/frontend restarted

F:\云计算技术\cloud-course-design>kubectl rollout status deployment/frontend
deployment "frontend" successfully rolled out

F:\云计算技术\cloud-course-design>kubectl get pods | findstr frontend
frontend-5cf5f9bdd5-zxvpn          1/1      Running   0           8s

F:\云计算技术\cloud-course-design>kubectl exec -it frontend-5cf5f9bdd5-zxvpn -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc:5001;
    }
}

```

图 2-11 修改 ConfigMap 后容器内配置文件更新

2.6 任务 6：HPA 弹性伸缩

后端 Deployment 配置 HPA，minReplicas=1，maxReplicas=4，目标 CPU 利用率为 60%。metrics-server 正常工作后，通过压测脚本和临时 CPU 计算负载提高后端 Pod CPU 使用率，HPA 根据 CPU 指标增加 backend 副本数。停止负载后，HPA 在稳定窗口后将副本数缩回最小值。

```

kubectl top nodes
kubectl apply -f k8s\backend-hpa.yaml
kubectl get hpa

```

```

# 监控窗口
kubectl get hpa -w
kubectl get pods -l app=backend -w

```

```

# Windows PowerShell 压测示例
$url = "http://<ELB-IP>/api/ping"
1..100 | ForEach-Object {
    Start-Job -ScriptBlock {
        param($u)
        for ($i = 0; $i -lt 200; $i++) {
            try { Invoke-WebRequest -UseBasicParsing -Uri $u | Out-Null } catch {}
        }
    } -ArgumentList $url
}

```

HPA 扩容存在一定延迟，主要由 metrics-server 采集周期、HPA 控制器评估间隔和新 Pod 镜像启动时间共同造成。缩容也存在稳定窗口，用于避免负载短时波动导致频繁扩缩容。HPA 的价值在于按需增加副本，在负载下降后释放资源，从而降低持续运行成本。

```
F:\云计算技术\cloud-course-design>kubectl get hpa
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa    Deployment/backend  cpu: 2%/60%      1         4         2         22s

F:\云计算技术\cloud-course-design>kubectl get pods | findstr backend
backend-6769558f75-csbc5          1/1      Running   1 (18m ago)   36m
```

图 2-12HPA 创建成功并获取 CPU 指标

```
F:\云计算技术\cloud-course-design>kubectl get hpa -w
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa    Deployment/backend  cpu: 5%/60%      1         4         2         3m21s
```

图 2-13 压测过程中 backend Pod 扩容

```
F:\云计算技术\cloud-course-design>kubectl get hpa -w
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa    Deployment/backend  cpu: 2%/60%      1         4         2         4m38s
```

图 2-14 停止负载后 backend Pod 缩容

```
F:\云计算技术\cloud-course-design>kubectl describe hpa backend-hpa
Name: backend-hpa
Namespace: default
Labels: <none>
Annotations: <none>
CreationTimestamp: Tue, 02 Jun 2026 02:28:24 +0800
Reference: Deployment/backend
Metrics:
  resource cpu on pods (as a percentage of request): 2% (1m) / 60%
Min replicas: 1
Max replicas: 4
Deployment pods: 1 current / 1 desired
Conditions:
  Type          Status Reason                               Message
  ----          -
AbleToScale    True  ReadyForNewScale                       recommended size matches current size
ScalingActive  True  ValidMetricFound                       the HPA was able to successfully calculate a replica count from cpu resour
ce utilization (percentage of request)
ScalingLimited False DesiredWithinRange                   the desired count is within the acceptable range
Events:
  Type          Reason          Age   From                               Message
  ----          -
Normal         SuccessfulRescale 58s   horizontal-pod-autoscaler         New size: 1; reason: All metrics below target
```

图 2-1 日志查看

由图可知，本次并没有出现扩容和缩容的情况，查阅日志得出结论：当前 backend Pod 的 CPU 使用率峰值只有 5%，而扩容阈值是 60%，所以 HPA 认为负载太低，甚至把副本数维持在最小值 1，主要原因是 /api/ping 太轻了。它只是 Flask 收请求+Redis INCR，大部分时间是网络 I/O，不怎么吃 CPU。而这个 HPA 监控的是 CPU 使用率，不是请求数、QPS 或响应时间。所以哪怕请求很多，只要 CPU 没上去，HPA 就不会扩容

3 第二部分：方向 A Spark 大数据分析

第二部分选择方向 A，在 CCE 集群上安装 Spark Operator，通过 SparkApplication 提交 PySpark 作业。数据集选择豆瓣电影评分数据集，字段包括 movie_id、title、original_title、year、rating_score、rating_count、genres、countries、directors、collect_count、summary 等。

3.1A-0: Spark Operator 环境部署

Spark Operator 通过本地 Helm Chart 安装，镜像替换为 SWR 中的 spark-operator:2.5.0。PySpark 基础镜像使用 swr.cn-north-4.myhuaweicloud.com/cloud-51/pyspark:v9。为 Driver 创建 default/sparkServiceAccount，并授予创建 Pod、Service、ConfigMap 以及 watch/list Pod 的权限。

```
helm install spark-op spark\spark-operator ^
--set image.registry=swr.cn-north-4.myhuaweicloud.com ^
--set image.repository=cloud-51/spark-operator ^
--set image.tag=2.5.0
```

```
kubectl get pods | findstr spark-op
kubectl get endpoints | findstr spark
kubectl create serviceaccount spark
kubectl create clusterrolebinding spark-admin-role --clusterrole=cluster-admin --
serviceaccount=default:spark
```

WordCount 作业用于验证 SparkApplication 提交流程。Driver Pod 进入 Completed 状态，Executor Pod 完成运行，说明 Spark Operator、PySpark 镜像、ServiceAccount 权限和 K8s 调度流程均可用。

```
1  apiVersion: sparkoperator.k8s.io/v1beta2
2  kind: SparkApplication
3  metadata:
4    name: pyspark-wordcount
5    namespace: default
6  spec:
7    type: Python
8    pythonVersion: "3"
9    mode: cluster
10   image: swr.cn-north-4.myhuaweicloud.com/cloud-51/pyspark:v9
11   imagePullPolicy: IfNotPresent
12   mainApplicationFile: local:///opt/spark/work/wordcount.py
13   sparkVersion: "3.4.0"
14   restartPolicy:
15     type: Never
16   driver:
17     cores: 1
18     memory: "1g"
19     serviceAccount: spark
20   executor:
21     cores: 1
22     instances: 2
23     memory: "1g"
```

图 3-1sparkapplication-wordcount.yaml

```
F:\云计算技术\cloud-course-design>kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
pyspark-wordcount-low-driver        1/1     Running   0           1s
spark-op-spark-operator-controller-6d4fd84b6f-l9szl  1/1     Running   0          3h51m
spark-op-spark-operator-webhook-5745f96d88-n8bnq    1/1     Running   0          3h51m
wordcount-9daade9e848dddba-exec-1  0/1     Pending   0           0s
wordcount-9daade9e848dddba-exec-1  0/1     Pending   0           0s
wordcount-9daade9e848dddba-exec-1  0/1     ContainerCreating  0           0s
wordcount-9daade9e848dddba-exec-1  1/1     Running   0           1s
wordcount-9daade9e848dddba-exec-1  1/1     Terminating  0          15s
wordcount-9daade9e848dddba-exec-1  1/1     Terminating  0          15s
wordcount-9daade9e848dddba-exec-1  0/1     Completed    0          15s
wordcount-9daade9e848dddba-exec-1  0/1     Completed    0          16s
wordcount-9daade9e848dddba-exec-1  0/1     Completed    0          16s
pyspark-wordcount-low-driver        0/1     Completed    0          30s
pyspark-wordcount-low-driver        0/1     Completed    0          31s
```

图 3-2WordCount Driver 与 Executor Pod 状态

```
C:\Windows\System32
_LOCAL, 7200 bytes)
26/06/01 18:59:29 INFO BlockManagerInfo: Added broadcast_5_piece0 in memory on 10.0.1.147:36027 (size: 5.9 KiB, free: 11
6.9 MiB)
26/06/01 18:59:29 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 1 to 10.0.1.147:46
794
26/06/01 18:59:29 INFO TaskSetManager: Finished task 0.0 in stage 6.0 (TID 8) in 101 ms on 10.0.1.147 (executor 1) (1/1)
26/06/01 18:59:29 INFO TaskSchedulerImpl: Removed TaskSet 6.0, whose tasks have all completed, from pool
26/06/01 18:59:29 INFO DAGScheduler: ResultStage 6 (runJob at PythonRDD.scala:179) finished in 0.107 s
26/06/01 18:59:29 INFO DAGScheduler: Job 2 is finished. Cancelling potential speculative or zombie tasks for this job
26/06/01 18:59:29 INFO TaskSchedulerImpl: Killing all running tasks in stage 6: Stage finished
26/06/01 18:59:29 INFO DAGScheduler: Job 2 finished: runJob at PythonRDD.scala:179, took 0.500330 s
Top 10 words: [('spark', 8), ('hello', 7), ('data', 4), ('and', 3), ('big', 3), ('kubernetes', 3), ('is', 2), ('cloud',
2), ('learning', 2), ('for', 2)]
26/06/01 18:59:29 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/06/01 18:59:29 INFO SparkUI: Stopped Spark web UI at http://pyspark-wordcount-low-0b2ddb9e848db45c-driver-svc.default
.svc:4040
26/06/01 18:59:29 INFO KubernetesClusterSchedulerBackend: Shutting down all executors
26/06/01 18:59:29 INFO KubernetesClusterSchedulerBackend$KubernetesDriverEndpoint: Asking each executor to shut down
26/06/01 18:59:29 WARN ExecutorPodsWatchSnapshotSource: Kubernetes client has been closed.
26/06/01 18:59:29 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/06/01 18:59:29 INFO MemoryStore: MemoryStore cleared
26/06/01 18:59:29 INFO BlockManager: BlockManager stopped
26/06/01 18:59:29 INFO BlockManagerMaster: BlockManagerMaster stopped
26/06/01 18:59:29 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/06/01 18:59:29 INFO SparkContext: Successfully stopped SparkContext
26/06/01 18:59:30 INFO ShutdownHookManager: Shutdown hook called
26/06/01 18:59:30 INFO ShutdownHookManager: Deleting directory /var/data/spark-cece035c-367b-4fd1-ae11-9d345b7cfe98/spar
k-bd4a28e6-7023-45f2-88da-57226bb80c9d
26/06/01 18:59:30 INFO ShutdownHookManager: Deleting directory /tmp/spark-5a991703-c4f0-4bf2-9eb7-c4279c2f1043
26/06/01 18:59:30 INFO ShutdownHookManager: Deleting directory /tmp/spark-19a089f4-ff5c-4730-b1af-7dd9b9c88528
```

图 3-3WordCount Driver 日志输出

3.2A-1：数据清洗

数据清洗使用 PySpark 读取豆瓣电影评分 CSV 数据，并设置 header、inferSchema、multiLine、quote、escape 等参数，避免 summary 字段中的换行导致 CSV 行解析错误。读取后打印 Schema 和前 5 行，并对全部字段统计缺失行数 and 缺失比例。

字段	清洗策略	原因
title	删除空值或空字符串记录	标题是后续 Top-N 和窗口函数分析的关键标识字段，缺失时记录不可解释
rating_score	转换为 double；转换失败或缺失的记录删除	评分是聚合、排序和性能测试的核心指标，必须为数值型
year	使用正则提取四位年份并转换为 int	原始年份可能以字符串或浮点形式存储，需要统一为年份维度
genres/countries/directors	缺失或空字符串填充为 Unknown	描述性字段缺失不影响核心评分分析，填充可保留样本量

清洗后生成 title_clean、rating_num、year_num、genre_clean、country_clean、director_clean 等规范字段，并注册为 Spark SQL 临时视图 movies，供后续统计分析使用。

```

1  apiVersion: sparkoperator.k8s.io/v1beta2
2  kind: SparkApplication
3  metadata:
4    name: pyspark-analysis
5    namespace: default
6  spec:
7    type: Python
8    pythonVersion: "3"
9    mode: cluster
10   image: swr.cn-north-4.myhuaweicloud.com/cloud-51/pyspark-analysis:v3
11   imagePullPolicy: Always
12   mainApplicationFile: local:///opt/spark/work/analysis.py
13   sparkVersion: "3.4.0"
14
15   sparkConf:
16     "spark.kubernetes.driver.request.cores": "200m"
17     "spark.kubernetes.executor.request.cores": "200m"
18     "spark.driver.memoryOverhead": "128m"
19     "spark.executor.memoryOverhead": "128m"
20     "spark.sql.shuffle.partitions": "4"
21
22   restartPolicy:
23     type: Never
24
25   driver:
26     cores: 1
27     coreLimit: "500m"
28     memory: "512m"
29     serviceAccount: spark
30
31   executor:
32     cores: 1
33     coreLimit: "500m"
34     instances: 1
35     memory: "512m"

```

图 3-4sparkapplication-analysis.yaml

```

F:\云计算技术\cloud-course-design>kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
spark-op-spark-operator-controller-6d4fd84b6f-l9szl  1/1     Running   0           4h15m
spark-op-spark-operator-webhook-5745f96d88-n8bnq    1/1     Running   0           4h15m
pyspark-analysis-driver              0/1     Pending   0           0s
pyspark-analysis-driver              0/1     Pending   0           0s
pyspark-analysis-driver              0/1     ContainerCreating  0           0s
pyspark-analysis-driver              1/1     Running   0           1s
doubananalysis-9fb72e9e84a3d534-exec-1  0/1     Pending   0           0s
doubananalysis-9fb72e9e84a3d534-exec-1  0/1     Pending   0           0s
doubananalysis-9fb72e9e84a3d534-exec-1  0/1     ContainerCreating  0           0s
doubananalysis-9fb72e9e84a3d534-exec-1  1/1     Running   0           1s
doubananalysis-9fb72e9e84a3d534-exec-1  1/1     Terminating    0           88s
doubananalysis-9fb72e9e84a3d534-exec-1  1/1     Terminating    0           88s
doubananalysis-9fb72e9e84a3d534-exec-1  0/1     Completed      0           89s
doubananalysis-9fb72e9e84a3d534-exec-1  0/1     Completed      0           89s
doubananalysis-9fb72e9e84a3d534-exec-1  0/1     Completed      0           89s
pyspark-analysis-driver              0/1     Completed      0          104s
pyspark-analysis-driver              0/1     Completed      0          105s

```

图 3-5Driver 与 Executor Pod 状态

```
===== Schema
=====

root
|-- movie_id: string (nullable = true)
|-- title: string (nullable = true)
|-- original_title: string (nullable = true)
|-- year: string (nullable = true)
|-- rating_score: string (nullable = true)
|-- rating_count: string (nullable = true)
|-- genres: string (nullable = true)
|-- countries: string (nullable = true)
|-- directors: string (nullable = true)
|-- collect_count: string (nullable = true)
|-- summary: string (nullable = true)
```

图 3-6 数据 Schema 输出

===== 前5行 =====

movie_id	title	original_title	year	rating_score	rating_count	genres	countries	directors	collect_count	summary
11292052	[肖申克的救赎]The Shawshank Redemption[1994.0]9.7 [1992071 [犯罪/剧情 [美国 [弗兰克·德拉邦特[2866850 [20世纪40年代末，小有成就的青年银行家安迪（蒂姆·罗宾斯 Tim Robbins 饰）因涉嫌杀害妻子及她的情人而锒铛入狱。在这座名为肖申克的监狱内，希望似乎毫无意义。将身监禁的岁月无趣地了安迪接下来却给他带来的人生。经过多年，安迪尝试接近狱中颇有声望的典狱长（摩根·弗里曼 Morgan Freeman 饰），请求对方帮自己搞来小锤子，以此为契机，二人逐渐熟络。安迪也仿佛在鱼龙混杂、罪恶横生、黑白颠倒的牢狱中找到属于自己的求生之道。他利用自身的专业知识，帮助监狱管理厘清逃税、洗钱钱，同时凭借与瑞德的交往在犯人中间也渐渐受到礼遇。表面看来，他已如瑞德那样对那堵高墙从憎恨转变为处之泰然，但是对自由的渴望仍促使他朝着心中的希望和目标前进。而关于其罪行的真相，似乎更使这一切向前推进了一步.....]									
11291546	[霸王别姬	[霸王别姬	1993.0]9.6	1486938	[剧情/爱情/同性]	中国大陆/中国香港	陈凯歌	2266871	[段小楼（张丰毅）与程蝶衣（张国荣）是一对从小一起长大的结拜兄弟，两人一个演生，一个饰旦，一向配合天衣无缝，尤其一出《霸王别姬》，更是誉满京城。为此，两人约定合演一辈子《霸王别姬》。但两人对戏剧与人生关系的理解有本质不同，段小楼深知戏非人生，程蝶衣则是人戏不分。	
11292720	[阿甘正传	[Forrest Gump	1994.0]9.5	1509559	[剧情/爱情	[美国	[罗伯特·泽米吉斯]	2435797	[阿甘（汤姆·汉克斯 饰）于二战结束后不久出生在美国南方阿拉巴马州一个闭塞的小镇，他先天弱智，智商只有75，然而他的妈妈是一个性格坚强的女性，她常常鼓励阿甘“傻人有傻福”，要他自强不息。	

only showing top 5 rows

图 3-7 原始数据前 5 行

```

原始数据总行数: 117156

===== 缺失值比例 =====
movie_id: 缺失 3 行, 缺失比例 0.0000
title: 缺失 45747 行, 缺失比例 0.3905
original_title: 缺失 46929 行, 缺失比例 0.4006
year: 缺失 49641 行, 缺失比例 0.4237
rating_score: 缺失 51564 行, 缺失比例 0.4401
rating_count: 缺失 48633 行, 缺失比例 0.4151
genres: 缺失 54331 行, 缺失比例 0.4637
countries: 缺失 49220 行, 缺失比例 0.4201
directors: 缺失 58206 行, 缺失比例 0.4968
collect_count: 缺失 49600 行, 缺失比例 0.4234
summary: 缺失 68669 行, 缺失比例 0.5861

===== 自动识别字段 =====
title_col = title
rating_col = rating_score
year_col = year
genre_col = genres
director_col = directors
country_col = None

===== 清洗结果 =====
清洗前行数: 117156
清洗后行数: 63780
删除行数: 53376

```

图 3-8 各字段缺失值比例

```

===== 清洗后统计信息 =====
+-----+-----+-----+
|summary| rating_num| year_num|
+-----+-----+-----+
| count|      63780|      61850|
| mean| 2.572470994042028|1998.3169442198869|
| stddev|17.902534460996826|19.543340259488467|
| min|         0.0|       1931|
| max|       2002.0|       2021|
+-----+-----+-----+

===== 清洗后前5行 =====
+-----+-----+-----+-----+-----+-----+
|title_clean |rating_num|year_num|genre_clean |director_clean |country_clean|
+-----+-----+-----+-----+-----+-----+
|这个杀手不太冷|9.4   |1994   |剧情/动作/犯罪|吕克·贝松   |Unknown   |
|美丽人生   |9.5   |1997   |剧情/喜剧/爱情|罗伯托·贝尼尼 |Unknown   |
|泰坦尼克号 |9.4   |1997   |剧情/爱情/灾难|詹姆斯·卡梅隆 |Unknown   |
|海上钢琴师 |9.3   |1998   |剧情/音乐   |朱塞佩·托纳多雷|Unknown   |
|机器人总动员 |9.3   |2008   |科幻/动画/冒险|安德鲁·斯坦顿 |Unknown   |
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

图 3-9 清洗前后行数对比

3.3A-2: SparkSQL 统计分析

Spark SQL 统计分析围绕电影类型、评分、年份和导演/地区等字段展开。查询结果在 Driver 日志中输出，并分别截图保存。

3.3.1 查询 1：按类型统计电影数量和平均评分

```
SELECT genre_clean AS genre,  
       COUNT(*) AS movie_count,  
       ROUND(AVG(rating_num), 2) AS avg_rating,  
       ROUND(MAX(rating_num), 2) AS max_rating,  
       ROUND(MIN(rating_num), 2) AS min_rating  
FROM movies  
GROUP BY genre_clean  
ORDER BY movie_count DESC  
LIMIT 20;
```

该查询使用 GROUP BY 按电影类型分组，统计各类型电影数量、平均评分、最高评分和最低评分。电影数量可以反映数据集中不同类型的样本分布，平均评分可用于比较不同类型电影的整体评价水平。样本量较大的类型具有更稳定的统计意义，样本量较小的类型则更容易受到少数高分或低分电影影响。

```
===== A-2 查询1：按类型统计电影数量和平均评分 =====  
+-----+-----+-----+  
|genre    |movie_count|avg_rating|  
+-----+-----+-----+  
|剧情     |10196     |2.49    |  
|Unknown  |5115     |3.3     |  
|喜剧     |3726     |1.54    |  
|恐怖     |3319     |0.81    |  
|剧情/爱情|1843     |3.04    |  
|动画     |1491     |2.28    |  
|动作     |1483     |1.15    |  
|喜剧/爱情|1341     |2.74    |  
|惊悚     |978      |1.9     |  
|剧情/喜剧|974      |2.71    |  
|爱情     |945      |2.29    |  
|惊悚/恐怖|766      |1.47    |  
|喜剧/剧情|710      |3.09    |  
|剧情/惊悚|565      |2.86    |  
|剧情/喜剧/爱情|558    |2.77    |  
|剧情/犯罪|548      |1.98    |  
|科幻     |545      |1.21    |  
|剧情/动作|481      |1.69    |  
|犯罪     |474      |0.64    |  
|音乐     |464      |1.89    |  
+-----+-----+-----+
```

图 3-10 查询 1：按类型聚合统计结果

3.3.2 查询 2：评分最高 Top 10 电影

```
SELECT title_clean AS title,  
       rating_num AS rating,  
       year_num AS year,  
       genre_clean AS genre,  
       country_clean AS country,  
       director_clean AS director  
FROM movies  
ORDER BY rating_num DESC  
LIMIT 10;
```

该查询使用 ORDER BY 和 LIMIT 选取评分最高的前 10 部电影。Top-N 查询适合快速发现数据集中评分最高的样本，同时也能验证 rating_score 转换为数值型后的排序是否正确。查询结果可结合年份、类型、国家和导演字段观察高分电影的分布特点。

===== A-2 查询2: 评分最高的Top 10电影 =====				
title	rating year genre			
Milan	2002.0 null Unknown			
London	2000.0 null Unknown			
but even better is the second disc	1969.0 null the video is often dark (and sometimes limited to a single camera			
听我说，请听我说	1958.0 null 35			
我的哥哥理查德	1957.0 null 45			
DVDs	516.0 null Unknown			
张国荣热情演唱会	9.8 2000 纪录片/音乐/歌舞			
唱游大世界王菲香港演唱会98-99	9.8 1999 纪录片/音乐			
是，大臣 1984圣诞特辑	9.8 1984 喜剧			
肖申克的救赎	9.7 1994 犯罪/剧情			

图 3-11 查询 2：评分最高 Top 10 电影

3.3.3 查询 3：按年份统计电影数量和平均评分

```
SELECT year_num AS year,
       COUNT(*) AS movie_count,
       ROUND(AVG(rating_num), 2) AS avg_rating
FROM movies
WHERE year_num IS NOT NULL
GROUP BY year_num
ORDER BY year_num
LIMIT 50;
```

该查询从时间维度统计各年份电影数量和平均评分。电影数量反映不同年代在数据集中覆盖程度，平均评分则反映不同年份电影评价水平的变化。由于不同年份样本量差异较大，分析时需要同时观察 movie_count 与 avg_rating，避免只依据平均值做结论。

===== A-2 查询3: 按年份统计电影数量和平均评分 =====		
[year]	[movie_count]	[avg_rating]
[1931]	1	9.3
[1936]	1	9.3
[1939]	1	9.3
[1940]	1	8.8
[1943]	42	1.22
[1944]	158	1.22
[1945]	155	1.54
[1946]	176	1.24
[1947]	193	1.5
[1948]	214	1.79
[1949]	208	1.52
[1950]	223	1.42
[1951]	244	1.57
[1952]	253	1.38
[1953]	307	1.33
[1954]	269	1.48
[1955]	276	1.62
[1956]	311	1.51
[1957]	349	1.41
[1958]	357	1.56
[1959]	399	1.25
[1960]	379	1.54
[1961]	341	1.41
[1962]	353	1.9
[1963]	167	1.55
[1965]	1	9.0
[1966]	322	1.86
[1967]	383	1.52
[1968]	439	1.42
[1969]	442	1.7
[1970]	459	1.51
[1971]	488	1.75
[1972]	472	1.55
[1973]	516	1.55
[1974]	523	1.49
[1975]	514	1.62
[1976]	526	1.67
[1977]	488	1.61
[1978]	520	1.37
[1979]	577	1.93
[1980]	611	1.77
[1981]	389	1.82
[1982]	4	5.63
[1983]	330	2.12
[1984]	566	2.39
[1985]	623	2.3
[1986]	615	2.35
[1987]	748	2.25
[1988]	703	2.26
[1989]	797	2.42

图 3-12 查询 3：按年份统计电影数量和平均评分

3.3.4 查询 4：窗口函数选取每年评分最高电影

```
SELECT year, title, rating, genre, director
FROM (
    SELECT year_num AS year,
           title_clean AS title,
           rating_num AS rating,
           genre_clean AS genre,
           director_clean AS director,
           ROW_NUMBER() OVER(PARTITION BY year_num ORDER BY rating_num DESC) AS rn
    FROM movies
    WHERE year_num IS NOT NULL
) t
WHERE rn = 1
ORDER BY year
LIMIT 50;
```

该查询使用 ROW_NUMBER() 窗口函数，在每个年份分组内部按评分降序排序，并取 rn=1 的记录。窗口函数的优势是既能完成“每组 Top 1”分析，又能保留标题、类型、导演等明细字段，相比普通 GROUP BY 更适合展示具体电影样本。

===== A-2 查询4：窗口函数，每年评分最高电影 =====				
year	title	rating	genre	
[1931]	城市之光	9.3	喜剧/剧情/爱情	
[1936]	摩登时代	9.3	剧情/喜剧/爱情	
[1939]	乱世佳人	9.3	剧情/历史/爱情	
[1940]	魂断蓝桥	8.8	剧情/爱情/战争	
[1943]	复仇之日	8.4	剧情/恐怖/爱情	
[1944]	双重赔偿	8.5	犯罪/黑色电影/惊悚	
[1945]	天堂的孩子	8.6	剧情/爱情	
[1946]	擦鞋童	8.6	剧情	
[1947]	凡尔杜先生	8.7	喜剧/犯罪/剧情	
[1948]	偷自行车的人	9.0	犯罪/剧情	
[1949]	哀乐中年	9.2	剧情/爱情/家庭	
[1950]	罗生门	8.8	Unknown	
[1951]	倒扣的王牌	8.7	剧情/黑色电影	
[1952]	生之欲	9.1	剧情	
[1953]	东京物语	9.2	剧情/家庭	
[1954]	十二怒汉 (电视版)	9.5	剧情/犯罪/悬疑	
[1955]	莫扎特 歌剧《唐乔瓦尼》 (唐璞)	9.0	音乐/喜剧/剧情	
[1956]	夜之河	8.9	剧情	
[1957]	控方证人	9.6	剧情/犯罪/悬疑	
[1958]	穆桂英挂帅	9.0	剧情	
[1959]	热情如火	8.8	喜剧	
[1960]	女人步上楼梯时	9.1	剧情	
[1961]	大闹天宫	9.3	动画/奇幻	
[1962]	红楼梦	9.0	剧情/爱情/古装	
[1963]	大逃亡	8.4	Unknown	
[1965]	音乐之声	9.0	剧情/传记/爱情	
[1966]	虎口脱险	8.9	Unknown	
[1967]	马拉·萨德	9.0	剧情/音乐/历史	
[1968]	2001太空漫游	8.8	科幻/惊悚/冒险	
[1969]	皮罗斯马尼	9.0	剧情/传记	
[1970]	同流者	8.6	Unknown	
[1971]	跑道终点	8.7	剧情	
[1972]	教父	9.3	剧情/犯罪	
[1973]	婚姻生活	9.0	剧情/爱情	
[1974]	教父2	9.2	剧情/犯罪	
[1975]	飞越疯人院	9.1	剧情	
[1976]	从容的快板	8.8	动画/喜剧/奇幻	
[1977]	史努比的惊险夏令营	9.1	动画/冒险/家庭	
[1978]	秋日奏鸣曲	9.0	剧情	
[1979]	哪吒闹海	9.1	冒险/动画/奇幻	
[1980]	影武者	9.0	剧情/历史/战争	
[1981]	鬼太鼓座	8.8	纪录片	
[1982]	E.T. 外星人	8.6	剧情/科幻	
[1983]	Abhilasha	9.3	惊悚	
[1984]	是，大臣 1984圣诞特辑	9.8	喜剧	
[1985]	天下一家	9.6	音乐	
[1986]	马斯格雷夫仪式	9.3	犯罪/悬疑	
[1987]	北国之恋：1987初恋	9.5	剧情	
[1988]	张国荣88演唱会	9.5	纪录片/音乐	
[1989]	张国荣告别演唱会	9.7	纪录片/音乐/歌舞	

图 3-13 查询 4：窗口函数每年评分最高电影

3.4A-3: 性能对比与 Amdahl 分析

性能对比选择 A-2 查询 1“按类型统计电影数量和平均评分”作为统一任务。该任务包含数据读取、字段清洗、分组聚合和排序操作，分别使用 Pandas 单机、PySpark 1 Executor 和 PySpark 2 Executors 执行，记录耗时并绘制对比图。

实现方式	运行环境	Executor 数	内存配置	耗时/s
Pandas	Windows 本地单机	-	16gb	0.3175
PySpark	CCE+Spark Operator	1	512Mi	5.5078
PySpark	CCE+Spark Operator	2	512Mi	3.8450

设 PySpark 1 Executor 的耗时为 T_1 ，PySpark 2 Executors 的耗时为 T_2 ，则实测加速比 $S=T_1/T_2$ ，并行效率 $E=S/2$ 。Amdahl 定律为 $S(N)=1/((1-P)+P/N)$ ，其中 P 为可并行比例。对于 $N=2$ ，可由 $P=2\times(1-1/S)$ 反推出近似可并行比例。

在本实验中，PySpark 增加 Executor 后不会线性加速。原因包括 Driver 初始化、Pod 调度、Executor 启动、DAG 构建、CSV 读取、序列化、Shuffle、结果收集等开销。数据规模较小时，分布式框架的固定开销占比较高；并且部分步骤只能由 Driver 负责或需要全局同步，因此实测加速比通常低于理论线性加速。

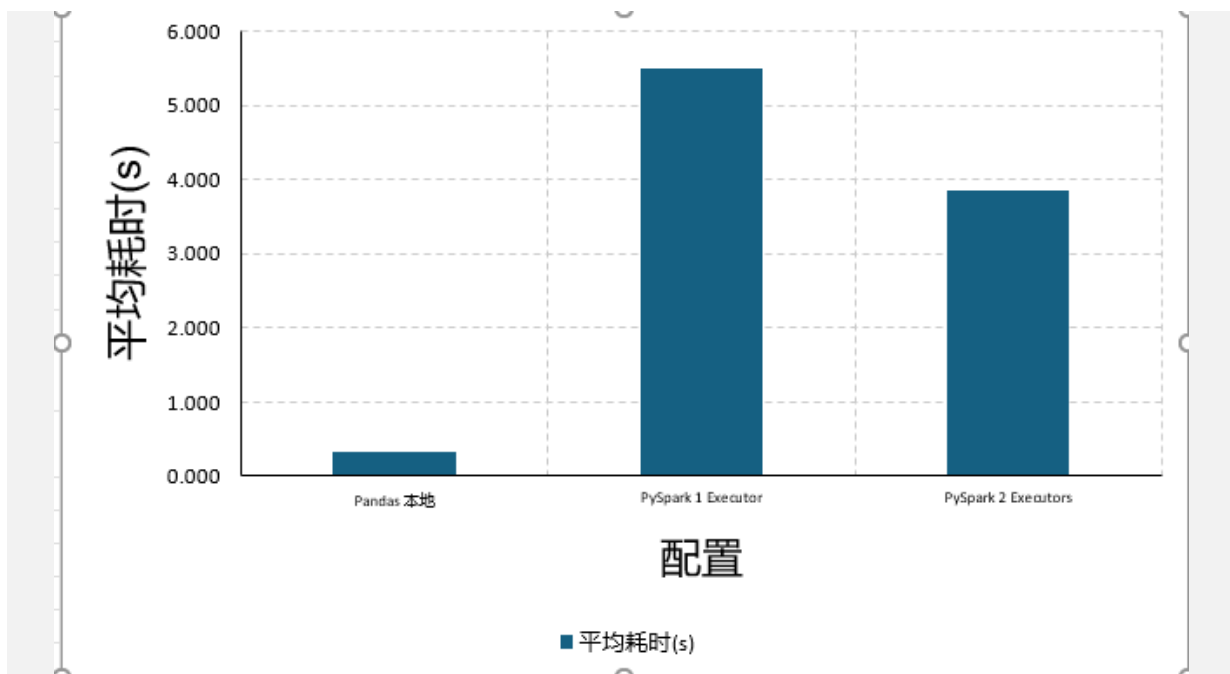


图 3-14 Pandas 与 PySpark 执行时间对比图

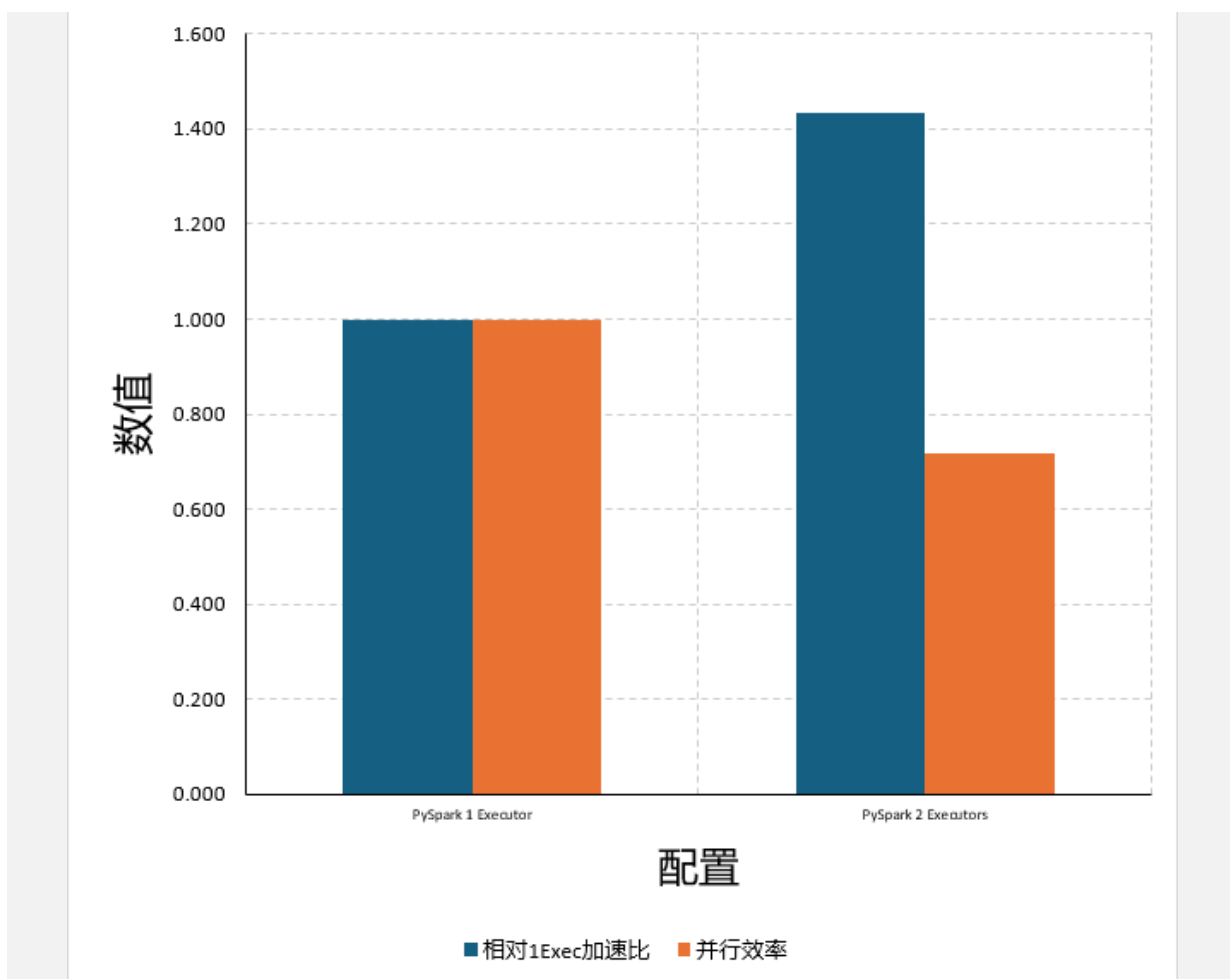


图 3-15 Amdahl 加速比计算表

4 总结与收获

本课程设计从容器镜像构建、镜像仓库推送、Kubernetes 资源编排、服务暴露、配置分离、持久化存储、弹性伸缩，到 Spark on Kubernetes 并行分析，完整串联了云计算平台搭建与并行数据处理流程。第一部分中，Flask 后端、Nginx 前端和 Redis 被拆分为不同容器并部署到 CCE，体现了容器化应用的模块化思想；ConfigMap、Secret、PVC 和 HPA 分别解决了配置管理、敏感信息管理、状态数据保存和弹性资源调度问题。第二部分中，Spark Operator 将 Spark 作业抽象为 Kubernetes 原生资源，使 Driver 和 Executor 可以由 Kubernetes 调度和管理，便于在云平台上完成分布式数据处理。

通过本次实践可以看到，云计算并不只是“把程序放到云上运行”，而是通过镜像、编排、网络、存储和自动伸缩等机制提高系统的可维护性和弹性。Spark 分析部分也说明，并行计算的性能提升受到数据规模、任务划分、通信开销和资源调度的共同影响。对于小规模数据，分布式框架的启动和调度开销可能抵消部分并行收益；对于更大规模数据，Executor 增加带来的收益才会更明显。整个实验加深了我对容器化部署、Kubernetes 资源对象、云平台负载均衡、持久化存储和 Spark 并行执行模型的理解。

5 附录：核心配置与代码片段

5.1 后端 Deployment 关键配置

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: swr.cn-north-4.myhuaweicloud.com/cloud-51/backend:v1
          ports:
            - containerPort: 5000
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
              cpu: "300m"
              memory: "256Mi"
          envFrom:
            - configMapRef:
                name: backend-config
          env:
            - name: REDIS_PASSWORD
              valueFrom:
```

```
secretKeyRef:
  name: redis-secret
  key: password
```

5.2 Redis PVC 关键配置

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: redis-data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: csi-disk
  resources:
    requests:
      storage: 10Gi

# Redis Deployment 中挂载
volumeMounts:
- name: redis-data
  mountPath: /data
volumes:
- name: redis-data
  persistentVolumeClaim:
    claimName: redis-data-pvc
```

5.3 SparkApplication 分析作业关键配置

```
apiVersion: sparkoperator.k8s.io/v1beta2
kind: SparkApplication
metadata:
  name: pyspark-analysis
spec:
  type: Python
  pythonVersion: "3"
  mode: cluster
  image: swr.cn-north-4.myhuaweicloud.com/cloud-51/pyspark-analysis:v4
  imagePullPolicy: Always
  mainApplicationFile: local:///opt/spark/work/analysis.py
  sparkVersion: "3.4.0"
  restartPolicy:
    type: Never
  driver:
    cores: 1
    memory: "512m"
    serviceAccount: spark
  executor:
    cores: 1
    instances: 2
    memory: "512m"
```

5.4 analysis.py 核心清洗逻辑

```
df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
```

```

.option("multiLine", True)
.option("quote", "")
.option("escape", "")
.csv(path))

clean_df = df.dropDuplicates()
clean_df = clean_df.filter(col("title").isNotNull() & col("rating_score").isNotNull())
clean_df = clean_df.withColumn("rating_num", col("rating_score").cast("double"))
clean_df = clean_df.filter(col("rating_num").isNotNull())
clean_df = clean_df.withColumn("year_num", regexp_extract(col("year").cast("string"),
r"(\d{4})", 1).cast("int"))
clean_df = clean_df.withColumn("genre_clean", when(col("genres").isNull(),
"Unknown").otherwise(trim(col("genres"))))
clean_df.createOrReplaceTempView("movies")

```